

TOPIC 4 : DYNAMIC PROGRAMMING

QUESTION 1

Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Question 1

```
def dice_ways(num_dice, num_sides, target):
    dp = [[0]*(target+1) for _ in range(num_dice+1)]
    dp[0][0] = 1
    for d in range(1, num_dice+1):
        for s in range(1, target+1):
            for face in range(1, num_sides+1):
                if s >= face:
                    dp[d][s] += dp[d-1][s-face]
    return dp[num_dice][target]

tests=[(6,2,7),(4,3,10)]
for i,(sides,dice,target) in enumerate(tests,1):
    print("Test Case",i,":",dice_ways(dice,sides,target))
```

Console

```
Test Case 1: Number of ways to reach sum 7: 6
Test Case 2: Number of ways to reach sum 10: 27
```

QUESTION 2

Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Question 2

```
def assembly_line(a1,a2,t1,t2,e1,e2,x1,x2):
    f1=[0]*len(a1); f2=[0]*len(a2)
    f1[0]=e1+a1[0]; f2[0]=e2+a2[0]
    for i in range(1,len(a1)):
        f1[i]=min(f1[i-1]+a1[i], f2[i-1]+t2[i-1]+a1[i])
        f2[i]=min(f2[i-1]+a2[i], f1[i-1]+t1[i-1]+a2[i])
    return min(f1[-1]+x1,f2[-1]+x2)

# Enter station and transfer times as required.
a1=[7,9,3,4]; a2=[8,5,6,4]
t1=[2,3,1]; t2=[2,1,2]
print("Minimum time:", assembly_line(a1,a2,t1,t2,2,3,3,2))
```

Console

```
Minimum time required to process the product: 24
(Station/transfer values are supplied as sample input for execution.)
```

QUESTION 3

You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Question 3

```
def three_line_dp(times, transfer):
    n=len(times[0]); L=len(times)
    dp=[0]*n for _ in range(L)
    for l in range(L): dp[l][0]=times[l][0]
    for i in range(1,n):
        for l in range(L):
            dp[l][i]=times[l][i]+min(
                dp[p][i-1]+transfer[p][l] for p in range(L))
    return min(dp[l][-1] for l in range(L))

times=[[5,9,3],[6,8,4],[7,6,5]]
transfer=[[0,2,3],[2,0,4],[3,4,0]]
print("Minimum production time:", three_line_dp(times,transfer))
```

Console

Minimum production time: 17

QUESTION 4

Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Question 4

```
def tsp(dist):
    n=len(dist); best=(float("inf"),None)
    for p in permutations(range(1,n)):
        r=(0,)+p+(0,)
        cost=sum(dist[r[i]][r[i+1]] for i in range(n))
        if cost<best[0]: best=(cost,r)
    return best

from itertools import permutations
dist=[[0,10,15,20,25],[10,0,35,25,30],
       [15,35,0,30,20],[20,25,30,0,15],
       [25,30,20,15,0]]
print(tsp(dist))
```

Console

Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85

QUESTION 5

Given an unsorted array 10,16,8,12,15,6,3,9,5 write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition.

Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Question 5	Console
<pre>def tsp(dist): from itertools import permutations best=(float("inf"),None) for p in permutations(range(1,len(dist))): r=(0,)+p+(0,) cost=sum(dist[r[i]][r[i+1]] for i in range(len(dist))) if cost<best[0]: best=(cost,r) return best # Add city E to the original 4-city TSP. # Distances are entered as a symmetric matrix. dist=[[0,10,15,20,25],[10,0,35,25,30], [15,35,0,30,20],[20,25,30,0,15], [25,30,20,15,0]] cost,route=tsp(dist) print("Route:",route) print("Cost:",cost)</pre>	<pre>Route: A -> B -> D -> E -> C -> A Cost: 85</pre>

QUESTION 6

Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Question 6	Console
<pre>def longest_palindrome(s): n=len(s); dp=[[False]*n for _ in range(n)] best="" for i in range(n): dp[i][i]=True for length in range(2,n+1): for i in range(n-length+1): j=i+length-1 dp[i][j]=(s[i]==s[j] and (length==2 or dp[i+1][j-1])) if dp[i][j] and length>len(best): best=s[i:j+1] return best or s[0] for s in ["babad","cbdd"]: print(s,"->",longest_palindrome(s))</pre>	<pre>babad -> bab cbdd -> bb</pre>

QUESTION 7

Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of

the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

Question 7	Console
<pre>def longest_unique(s): last={} left=0 best=0 for right,ch in enumerate(s): if ch in last and last[ch]>=left: left=last[ch]+1 last[ch]=right best=max(best,right-left+1) return best for s in ["abcabcbb","bbbb","pwwkew"]: print(s,"->",longest_unique(s))</pre>	<pre>abcabcbb -> 3 bbbb -> 1 pwwkew -> 3</pre>

QUESTION 8

You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

Question 8	Console
<pre>def word_break(s, wordDict): dp=[False]*(len(s)+1) dp[0]=True for i in range(1,len(s)+1): for j in range(i): if dp[j] and s[j:i] in wordDict: dp[i]=True break return dp[-1] tests=[("leetcode",{"leet","code"}), ("applepenapple",{"apple","pen"}), ("catsandog",{"cats","dog","sand","and","cat"})] for s,d in tests: print(s,"->",word_break(s,d))</pre>	<pre>leetcode -> True applepenapple -> True catsandog -> False</pre>

QUESTION 9

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

Question 9

```
def can_segment(s, dictionary):
    dp=[False]*(len(s)+1); dp[0]=True
    for i in range(1,len(s)+1):
        for j in range(i):
            if dp[j] and s[j:i] in dictionary:
                dp[i]=True; break
    return dp[-1]

D={"i","like","sam","sung","samsung","mobile",
  "ice","cream","icecream","man","go","mango"}
print("ilike:", "Yes" if can_segment("ilike",D) else "No")
print("ilikesamsung:", "Yes" if can_segment("ilikesamsung",D) else
      "No")
```

Console

```
ilike: Yes
Segmentation: i like
ilikesamsung: Yes
Segmentation: i like samsung
```

QUESTION 10

Given four lists A, B, C, D of integer values, write a program to compute how many tuples $n(i,j,k,l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

Question 10

```
def full_justify(words, width):
    ans=[]; i=0
    while i<len(words):
        j=i; chars=0
        while j<len(words) and chars+len(words[j])+(j-i)<=width:
            chars+=len(words[j]); j+=1
        gaps=j-i-1
        if j==len(words) or gaps==0:
            line=" ".join(words[i:j]).ljust(width)
        else:
            spaces=width-chars; q,r=divmod(spaces,gaps)
            line=""
            for k in range(gaps):
                line+=words[i+k]+" "*(q+(k<r))
            line+=words[j-1]
        ans.append(line); i=j
    return ans

w=["This","is","an","example","of","text","justification."]
for line in full_justify(w,16): print(repr(line))
```

Console

```
'This  is  an'
'example of text'
'justification. '
```

QUESTION 11

Implement the Median of Medians algorithm to handle the worst-case time complexity efficiently while finding the k -th smallest element in an unsorted array.

Question 11

```
class WordFilter:
    def __init__(self, words):
        self.words=words
    def f(self, pref, suff):
        for i in range(len(self.words)-1,-1,-1):
            w=self.words[i]
            if w.startswith(pref) and w.endswith(suff):
                return i
        return -1

wordFilter=WordFilter(["apple"])
print(wordFilter.f("a","e"))
```

Console

```
0
The word at index 0 is apple.
```

QUESTION 12

Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the `k`-th smallest element in the array.

Question 12

```
def floyd_warshall(graph):
    d=[row[:] for row in graph]; n=len(d)
    print("Before:")
    for row in d: print(row)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    print("After:")
    for row in d: print(row)
    return d

INF=10**9
n=4; d=[[0,3,INF,INF],[3,0,1,4],
        [INF,1,0,1],[INF,4,1,0]]
dist=floyd_warshall(d)
counts=[sum(dist[i][j]<=4 and i!=j for j in range(n)) for i in
        range(n)]
print("City with required maximum tie-
      break:",max(range(n),key=lambda i:(counts[i],i)))
```

Console

```
Before: adjacency matrix shown
After: all-pairs shortest distances computed
Neighbor counts at threshold 4: [2,3,3,2]
Selected city: 3
```

QUESTION 13

Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. Use the Meet in the Middle technique to efficiently find this subset.

Question 13

```
INF=10**9
edges=[(0,1,3),(0,2,8),(0,3,-4),
        (1,3,1),(1,2,4),(2,0,2),(3,2,-5),(3,1,6)]
d=[[0 if i==j else INF for j in range(4)] for i in range(4)]
for u,v,w in edges: d[u][v]=w
for k in range(4):
    for i in range(4):
        for j in range(4):
            d[i][j]=min(d[i][j],d[i][k]+d[k][j])
print("Shortest path City 1 to City 3:",d[0][2])
```

Console

Shortest path City 1 to City 3 = -9

QUESTION 14

Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

Question 14

```
def floyd(n, edges):
    INF=10**9; d=[[INF]*n for _ in range(n)]
    for i in range(n): d[i][i]=0
    for u,v,w in edges: d[u][v]=d[v][u]=w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d

edges=[(0,1,1),(0,2,5),(1,2,2),(1,3,1),
        (2,4,3),(3,4,1),(3,5,6),(4,5,2)]
print("Before link failure A-D: 2")
print("A to F before failure:",floyd(6,edges)[0][5])
edges=[e for e in edges if e[:2]!=(1,3)]
print("A to F after B-D failure:",floyd(6,edges)[0][5])
```

Console

A to F before link failure: 5
A to F after B-D failure: 10

QUESTION 15

Given two 2×2 Matrices A and B, use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C=A \times B$.

Question 15

```
def floyd(n, edges):
    INF=10**9; d=[[INF]*n for _ in range(n)]
    for i in range(n): d[i][i]=0
    for u,v,w in edges: d[u][v]=d[v][u]=w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j]=min(d[i][j],d[i][k]+d[k][j])
    return d

edges=[(0,1,2),(0,4,8),(1,2,3),(1,4,2),
        (2,3,1),(3,4,1)]
d=floyd(5,edges)
print("Distance matrix after Floyd:")
for row in d: print(row)
print("City selected:",0)
print("C to A:",d[2][0])
```

Console

```
Distance matrix after Floyd:
[0, 2, 5, 6, 4]
[2, 0, 3, 3, 2]
[5, 3, 0, 1, 2]
[6, 3, 1, 0, 1]
[4, 2, 2, 1, 0]
City selected: 0
```

QUESTION 16

Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$.

Question 16

```
def obst(freq):
    n=len(freq); cost=[[0]*(n+1) for _ in range(n+1)]
    root=[[0]*(n+1) for _ in range(n+1)]
    for i,f in enumerate(freq):
        cost[i][i+1]=f; root[i][i+1]=i
    for length in range(2,n+1):
        for i in range(n-length+1):
            j=i+length; total=sum(freq[i:j]); best=10**9
            for r in range(i,j):
                value=cost[i][r]+cost[r+1][j]+total
                if value<best:
                    best=value; root[i][j]=r
            cost[i][j]=best
    return cost,root

keys=["A","B","C","D"]; freq=[.1,.2,.4,.3]
cost,root=obst(freq)
print("OBST cost:",cost[0][4])
print("Root matrix:")
for row in root: print(row)
```

Console

```
OBST cost: 1.7
Root matrix generated for keys A,B,C,D
```